

Manual do Programador (MP) — Analisador Semântico

Índice

- 1. [Visão geral técnica](#)
 - 2. [Arquitectura de módulos](#)
 - 3. [A Árvore de Sintaxe Abstrata \(AST\)](#)
 - 4. [O Analisador Semântico](#)
 - 5. [Estruturas de dados](#)
 - 6. [Funções auxiliares](#)
 - 7. [Lógica de cada verificação semântica](#)
 - 8. [Inferência de tipos](#)
 - 9. [Gestão de escopos](#)
 - 10. [Problemas e soluções](#)
 - 11. [Integração com o main](#)
 - 12. [Tabela de símbolos](#)
 - 13. [Testes](#)
 - 14. [Referências](#)
-

1. Visão geral técnica

O analisador semântico é a terceira fase do compilador. Recebe a AST (Árvore de Sintaxe Abstrata) produzida pelo parser e percorre-a recursivamente, validando:

- 1. **Declaração** — variáveis, funções e classes estão declaradas antes de serem usadas
- 2. **Escopo** — cada nome é válido apenas no escopo onde foi declarado (e escopos ancestrais)
- 3. **Tipos** — operações e atribuições usam tipos compatíveis
- 4. **Estruturas de controlo** — break/continue em contextos válidos, return com tipo correcto

O analisador funciona num **único passe** (single-pass) sobre a AST, populando a tabela de símbolos e decorando cada nó com o seu `inferred_type`.

Ponto de entrada: `decorar_ast(AstNode) → (AstNode, Vec<Scope>)`

Ficheiro principal: `src/analise_semantica.rs` (729 linhas)

2. Arquitectura de módulos

3.3 Construtores — com exemplos concretos

Regra geral: nós internos (que têm filhos) não usam `token`. Nós folha (sem filhos) armazenam o lexema em `token`.

`AstNode::new(kind, line)` — nó vazio

Nó sem `token` e sem filhos. Usado para nós que são apenas marcadores estruturais ou para erro.

```
// break;    → nó folha sem token textual
AstNode::new(AstKind::Break, 5)

// continue; → nó folha sem token textual
AstNode::new(AstKind::Continue, 8)

// return;   (sem expressão) → nó com children vazio
let mut node = AstNode::new(AstKind::Return, 12);
// children é [], sem expressão de retorno
```

`AstNode::leaf(kind, token, line)` — nó folha

Nó sem filhos, com o lexema armazenado em `token`. Usado para terminais: identifiers, tipos, modificadores, literais.

```
// int x;
//    → Type folha com token = "int"
AstNode::leaf(AstKind::Type, "int", 3)

//    → Identifier folha com token = "x"
AstNode::leaf(AstKind::Identifier, "x", 3)

// public: → Modifier folha com token = "public"
AstNode::leaf(AstKind::Modifier, "public", 2)

// 42 → Literal folha com token = "42"
AstNode::leaf(AstKind::Literal, "42", 7)
```

`AstNode::with_children(kind, children, line)` — nó interno

Nó com lista de filhos mas **sem token**. Usado para constructos estruturais cujo significado vem da estrutura, não de um lexema.

```
// { stmt1; stmt2; } → Block com filhos
AstNode::with_children(AstKind::Block, vec![stmt1, stmt2], 10)

// int x, y, z; → Program com filhos (includes + declarações)
AstNode::with_children(AstKind::Program, children, 1)
```

```
// if (cond) { ... } else { ... }
// → If com 3 filhos: [condição, bloco_then, bloco_else]
AstNode::with_children(AstKind::If, vec![cond, then_block, else_block], 15)

// func(a, b)
// → Call com 3 filhos: [Identifier("func"), Identifier("a"), Identifier("b")]
AstNode::with_children(AstKind::Call, vec![func_id, arg_a, arg_b], 20)

// int soma(int a, int b) { ... }
// → FunctionDecl com filhos: [Type, Identifier, ParamList, Block]
AstNode::with_children(AstKind::FunctionDecl, vec![
    AstNode::leaf(AstKind::Type, "int", 1),
    AstNode::leaf(AstKind::Identifier, "soma", 1),
    param_list,
    body_block,
], 1)
```

AstNode::with_token(kind, children, token, line) — nó interno com token

Nós com filhos e um token que identifica a operação. Usado para operadores.

```
// a + b → BinaryOp com token = "+", filhos = [a, b]
AstNode::with_token(AstKind::BinaryOp, vec![id_a, id_b], "+", 5)

// x = 10 → Assign com token = "=", filhos = [x, literal_10]
AstNode::with_token(AstKind::Assign, vec![id_x, lit_10], "=", 5)

// !flag → UnaryOp com token = "!", filhos = [flag]
AstNode::with_token(AstKind::UnaryOp, vec![id_flag], "!", 5)

// i++ → UnaryOp com token = "post++", filhos = [i]
AstNode::with_token(AstKind::UnaryOp, vec![id_i], "post++", 5)
```

AstNode::with_original_token(kind, token, original_token, line) — literal preservado

Nó folha que preserva a variante original do `Token` enum. Usado apenas para literais que precisam de distinção semântica.

```
// true → Literal com token = "true", original_token = Token::TrueLiteral
AstNode::with_original_token(AstKind::Literal, "true", Token::TrueLiteral, 3)

// 3.14 → Literal com token = "3.14", original_token = Token::Float
AstNode::with_original_token(AstKind::Literal, "3.14", Token::Float, 7)

// 'a' → Literal com token = "'a'", original_token = Token::Char
AstNode::with_original_token(AstKind::Literal, "'a'", Token::Char, 9)

// 42 → Literal com token = "42", original_token = Token::Inteiro
AstNode::with_original_token(AstKind::Literal, "42", Token::Inteiro, 5)
```

Por que isto é necessário? Sem `original_token`, o analisador semântico só tem `"true"` em `token` — não saberia se é `Token::TrueLiteral` (bool) ou se é uma string `"true"`. O `original_token` preserva a informação do lexer.

3.4 Árvore concreta: exemplo completo

Para `int x = 10;` declarado dentro de uma função:

```
VarDecl
├─ Type           (leaf: token = "int")
├─ Identifier     (leaf: token = "x")
└─ Literal       (with_original_token: token = "42", original_token = Inteiro)
```

O parser chama:

```
AstNode::with_children(AstKind::VarDecl, vec![
    AstNode::leaf(AstKind::Type, &tipo, line),
    AstNode::leaf(AstKind::Identifier, &nome, line),
    AstNode::with_original_token(AstKind::Literal, &lex, original_token, line),
], line)
```

Depois da análise semântica, o nó `VarDecl` fica decorado:

```
VarDecl  [inferred_type = "int"]
├─ Type           [inferred_type = "int"]
├─ Identifier     [inferred_type = ""]
└─ Literal       [inferred_type = "int"] ← inferido a partir de original_token
```

3.5 Campos adicionados na Fase 3

- `original_token: Option<Token>` — necessário para o analisador semântico distinguir `true/false` (bool) de `1` (int) nos literais
- `inferred_type: String` — preenchido bottom-up durante a análise semântica
- `line: usize` — preenchido pelo parser para todas as chamadas a `AstNode`
- `ArrayDimension` — nó que encapsula as dimensões de um array

4. O Analisador Semântico

4.1 Ponto de entrada

```
pub fn decorar_ast(mut ast: AstNode) -> (AstNode, Vec<Scope>)
```

Cria um `SemanticAnalyzer` com um escopo global vazio (nível 0) e chama `semantic_analysis` recursivamente. Retorna a AST decorada e a lista de escopos.

4.2 Função principal

```
fn semantic_analysis(node: &mut AstNode, analyzer: &mut SemanticAnalyzer)
```

Função recursiva que percorre a AST. Para cada tipo de nó, aplica as verificações semânticas apropriadas e propaga o `inferred_type` de baixo para cima.

4.3 Padrão geral

1. processar filhos recursivamente (bottom-up)
2. validar regras semânticas para este nó
3. propagar `inferred_type` para o nó pai

Excepção: nós que criam escopos (Block, For) processam os filhos *dentro* do novo escopo.

5. Estruturas de dados

5.1 SymbolType (src/scope.rs)

```
pub enum SymbolType {  
    Int(&'static str),    // "int"  
    Float(&'static str),  // "float"  
    Double(&'static str), // "double"  
    Char(&'static str),    // "char"  
    Bool(&'static str),    // "bool"  
    String(&'static str),  // "string"  
}
```

5.2 SymbolCategory (src/scope.rs)

```
pub enum SymbolCategory {  
    Variable,  
    Function,  
    Class,  
    Array,  
    Parameter,  
}
```

5.3 BlockType (src/scope.rs)

Rastreia o constructo actual para validação de break/continue:

```
pub enum BlockType {  
    None,                // fora de qualquer constructo
```

```

Class(String),           // dentro de uma classe (nome da classe)
Function(String),        // dentro de uma função (nome da função)
Constructor(String),     // dentro de um construtor
For,
While,
DoWhile,
If,
Switch,
}

```

5.4 Symbol (src/scope.rs)

```

pub struct Symbol {
    pub name: String,
    pub symbol_type: SymbolType,
    pub category: SymbolCategory,
    pub scope_level: usize,
    pub line_declared: usize,
    pub memory_address: usize,
    pub size_in_bytes: usize,
    pub dimension: usize,           // 0 para não-arrays
    pub parameter_types: Vec<String>, // tipos dos parâmetros (funções)
}

```

5.5 Scope (src/scope.rs)

```

pub struct Scope {
    pub symbols: HashMap<String, Symbol>,
    pub parent_idx: Option<usize>, // índice do escopo pai (-1 = global)
    pub scope_level: usize,        // profundidade (0 = global)
    pub next_memory_offset: usize, // próximo endereço livre
}

```

5.6 SemanticAnalyzer (src/scope.rs)

```

pub struct SemanticAnalyzer {
    pub scopes: Vec<Scope>,
    pub current_scope_idx: usize,
    pub pending_parameters: Vec<Symbol>,
    pub block_type: BlockType,
    pub current_function_return_type: Option<String>,
    pub for_init_mode: bool,
}

```

Campo

scopes

current_scope_idx

Função

Vector de escopos (árvore de escopos representada como vector + parent_idx)

Índice do escopo actual

Campo	Função
<code>pending_parameters</code>	Parâmetros/variáveis de for que aguardam inserção no próximo Block
<code>block_type</code>	Constructo actual (para break/continue/constructor) — save/restore
<code>current_function_return_type</code>	Tipo de retorno da função actual (para Return) — save/restore
<code>for_init_mode</code>	Quando <code>true</code> , <code>VarDecl</code> empurra para <code>pending_parameters</code> em vez de inserir no escopo

6. Funções auxiliares

6.1 `tamanho_do_tipo(tipo: &str) → usize`

Retorna o tamanho em bytes de um tipo primitivo:

```
int → 4, float → 4, double → 8, char → 1, bool → 1, string → 8, default → 4
```

6.2 `tipo_para_symbol_type(tipo: &str) → SymbolType`

Converte string de tipo para `SymbolType`:

```
"int" → Int("int"), "float" → Float("float"), "double" → Double("double"), etc.
```

6.3 `symbol_type_to_string(st: &SymbolType) → String`

Converte `SymbolType` de volta para string — extrai o `&str` interno.

6.4 `lookup(analyzer, nome) → Option<&Symbol>`

Busca um nome subindo a hierarquia de escopos:

```
fn lookup(analyzer: &SemanticAnalyzer, nome: &str) -> Option<&Symbol> {
    let mut idx = Some(analyzer.current_scope_idx);
    while let Some(i) = idx {
        if let Some(sym) = analyzer.scopes[i].symbols.get(nome) {
            return Some(sym);
        }
        idx = analyzer.scopes[i].parent_idx;
    }
    None
}
```

Exemplo concreto — busca de "x" :

```
Escopo 2 (nivel 2): { temp: Variable }      ← current_scope_idx = 2
parent_idx → 1
```

```
Escopo 1 (nivel 1): { x: Variable, y: Variable }  ← parent_idx = 0
```



```
parent_idx → 0
```

```
Escopo 0 (nivel 0): { main: Function, soma: Function } ← parent_idx = None
```

```
lookup(analyzer, "x") :
```

1. Procura no escopo 2 → não encontra "x"
2. Sobe para escopo 1 (parent_idx = 1) → encontra "x" ✓

```
lookup(analyzer, "z") :
```

1. Procura no escopo 2 → não encontra "z"
2. Sobe para escopo 1 → não encontra "z"
3. Sobe para escopo 0 → não encontra "z"
4. parent_idx = None → retorna None

Ordem de prioridade: o lookup sempre encontra primeiro a variável do escopo mais interno (shadowing).

6.5 promover_tipo(a, b) → Option<String>

Promoção de tipos compatíveis. Retorna o tipo resultado se a conversão é possível:

```
a == b          → Some(a)
int ↔ float     → Some("float")
int ↔ double    → Some("double")
float ↔ double  → Some("double")
incompatíveis   → None
```

Nota: A promoção é simétrica e usa &str para comparação.

7. Lógica de cada verificação semântica

7.1 VarDecl (linhas 97–168)

1. Extrair tipo e nome dos filhos Type e Identifier
2. Se for_init_mode == true:
 - empurrar Symbol para pending_parameters (não inserir no escopo actual)
 - return
3. Verificar duplicata no escopo actual → panic!
4. Criar Symbol e inserir no escopo actual
5. Processar o inicializador recursivamente (se existir)
6. Verificar compatibilidade tipo_decl ↔ tipo_init → panic!
7. Propagar inferred_type = tipo

Caminho normal (int x = 10;):

```
// O parser cria:
AstNode::with_children(AstKind::VarDecl, vec![
    AstNode::leaf(AstKind::Type, "int", 5),
    AstNode::leaf(AstKind::Identifier, "x", 5),
    AstNode::with_original_token(AstKind::Literal, "10", Token::Inteiro, 5),
], 5)

// O analisador semântico:
// 1. Extrai tipo = "int", nome = "x"
// 2. Verifica que "x" não existe no escopo actual
// 3. Insere Symbol { name: "x", symbol_type: Int("int"), ... }
// 4. Processa o Literal: inferred_type = "int"
// 5. Verifica: promover_tipo("int", "int") = Some("int") ✓
// 6. node.inferred_type = "int"
```

Caminho do for-init (`for(int i = 0; ...)`):

```
// O mesmo VarDecl é criado, mas o analyzer tem for_init_mode = true
// 1. Extrai tipo = "int", nome = "i"
// 2. Vê for_init_mode == true → empurra para pending_parameters
// 3. NÃO insere no escopo actual (que pode ser o escopo da função, não do for)
// 4. return (sem processar o initializer)
```

Nota: O problema do nome ser sobrescrito pelo último Identifier no loop do parser ainda existe (pre-existente).

7.2 ArrayDecl (linhas 169–237)

1. Extrair tipo, nome e contar dimensões
2. Verificar duplicata → panic!
3. Para cada ArrayDimension:
 - processar expressão recursivamente
 - verificar que tipo é "int" → panic!
 - multiplicar total_elementos pelo valor literal
4. Calcular tamanho total = base_size × total_elementos
5. Criar Symbol com category = Array
6. Propagar inferred_type = tipo base

7.3 Param (linhas 243–267)

1. Extrair tipo e nome
2. Criar Symbol com category = Parameter
3. Empurrar para pending_parameters (não inserir no escopo actual!)
4. Propagar inferred_type = tipo

Razão: Os parâmetros não existem no escopo do parser; são inseridos no escopo do Body da função quando o Block cria o novo escopo.

7.4 FunctionDecl (linhas 307–363)

1. Extrair nome e tipo de retorno
2. Verificar duplicata → panic!
3. Extrair `parameter_types` dos `Params`
4. Inserir `Symbol` com `category = Function` e `parameter_types`
5. Guardar e actualizar `block_type` → `BlockType::Function`
6. Guardar e actualizar `current_function_return_type` → `Some(tipo)`
7. Processar filhos recursivamente
8. Restaurar `previous_ret` e `previous_block_type`
9. Propagar `inferred_type` = tipo

7.5 ConstructorDecl (linhas 364–430)

1. Extrair nome
2. Verificar que `block_type` actual é `Class` e nome == `class_name` → panic!
3. Verificar que está dentro de uma classe (`block_type` é `Class`) → panic!
4. Verificar duplicata → panic!
5. Guardar `block_type` → `BlockType::Constructor`
6. Guardar `current_function_return_type` → `Some("void")`
7. Processar filhos
8. Restaurar `previous_ret` e `block_type`

7.6 ClassDecl (linhas 268–306)

1. Extrair nome
2. Verificar duplicata → panic!
3. Inserir `Symbol` com `category = Class`
4. Guardar `block_type` → `BlockType::Class(nome)`
5. Processar filhos
6. Restaurar `block_type`

7.7 For (linhas 431–478)

1. Guardar e actualizar `block_type` → `BlockType::For`
2. Criar novo escopo (nível + 1)
3. `for_init_mode = true`
4. Processar `initializer` recursivamente (`VarDecl` empurra para `pending_parameters`)
5. `for_init_mode = false`
6. Drenar `pending_parameters` → inserir no novo escopo
7. Processar `condition` e `body` recursivamente
8. Verificar `condition` é `bool` ou `int` → panic!
9. Restaurar escopo anterior e `block_type`

Razão da criação antecipada de escopo: O `initializer` do `for ( int i = 0 )` declara `i`, que só deve ser visível dentro do `body`. O escopo é criado antes de processar o `initializer`, que usa `for_init_mode` para empurrar para `pending_parameters`, que depois são drenados para o novo escopo.

7.8 While (linhas 479–491)

1. Guardar `block_type` → `BlockType::While`
2. Processar filhos recursivamente
3. Verificar `condition` é `bool` ou `int` → `panic!`
4. Restaurar `block_type`

7.9 DoWhile (linhas 492–504)

1. Guardar `block_type` → `BlockType::DoWhile`
2. Processar filhos recursivamente
3. Verificar `condition` (`filho[1]`) é `bool` ou `int` → `panic!`
4. Restaurar `block_type`

Nota: Em `do-while`, o `body` é `filho[0]` e a `condition` é `filho[1]`.

7.10 If (linhas 505–517)

1. Guardar `block_type` → `BlockType::If`
2. Processar filhos recursivamente
3. Verificar `condition` é `bool` ou `int` → `panic!`
4. Restaurar `block_type`

7.11 Switch (linhas 518–525)

1. Guardar `block_type` → `BlockType::Switch`
2. Processar filhos recursivamente
3. Restaurar `block_type`

7.12 Break (linhas 526–531)

1. Verificar `block_type` é `For` | `While` | `DoWhile` | `Switch` → `panic!`

7.13 Continue (linhas 532–537)

1. Verificar `block_type` é `For` | `While` | `DoWhile` → `panic!`

7.14 Identifier (linhas 538–548)

1. Chamar `lookup(analyzer, nome)`
2. Se encontrado: `inferred_type = symbol_type_to_string(sym.symbol_type)`
3. Se não: `panic! ("Variavel 'x' nao declarada na linha Y")`

7.15 Literal (linhas 549–570)

Usa `original_token` para inferir o tipo exacto do literal:

```
TrueLiteral | FalseLiteral → "bool"
String → "string"
Char → "char"
Float → "double"          // literais com ponto flutuante são double por padrão
Inteiro → "int"
_ → "int"                  // fallback
```

7.16 Assign (linhas 571–594)

1. Processar filhos recursivamente
2. Verificar lvalue (`Identifier`, `Index`, `MemberAccess`, `PtrAccess`) → panic!
3. Verificar que `Identifier` está declarado → panic!
4. Verificar compatibilidade `tipo_esq` ↔ `tipo_dir` via `promover_tipo` → panic!
5. Propagar `inferred_type` = `tipo_esq`

7.17 BinaryOp (linhas 595–617)

1. Processar filhos recursivamente
2. Extrair `tipo_esq`, `tipo_dir` e operador
3. Se operador relacional (`==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`):
→ verificar compatibilidade via `promover_tipo` → panic!
→ `inferred_type` = "bool"
4. Senão:
→ `promover_tipo(tipo_esq, tipo_dir)` → panic se incompatível
→ `inferred_type` = tipo resultado

7.18 UnaryOp (linhas 618–638)

1. Processar filhos recursivamente
2. Extrair tipo do operando e operador
3. `"-"`, `"++"`, `"--"`, `"post++"`, `"post--"` → `inferred_type` = tipo
4. `!"` → `inferred_type` = "bool"
5. `"&"`, `"*"` → `inferred_type` = tipo (ponteiro/referência)

7.19 Call (linhas 639–675)

1. Processar filhos recursivamente
2. Extrair nome do `Identifier`
3. `lookup` → panic! se não encontrada
4. Verificar `category == Function` → panic!
5. Extrair tipos dos argumentos enviados
6. Comparar número de argumentos vs parâmetros → panic!

7. Para cada argumento: `promover_tipo(enviado, esperado) → panic!`
8. Propagar `inferred_type` = tipo de retorno da função

Exemplo concreto — `soma(x, 10)` :

```
CALL
├─ Identifier ("soma")      ← callee
├─ Identifier ("x")        ← argumento 1
└─ Literal (10)           ← argumento 2
```

O que acontece:

1. Processa `Identifier("soma")` → lookup **encontra** `soma` no escopo, `inferred_type = "int"`
2. Processa `Identifier("x")` → lookup **encontra** `x`, `inferred_type = "int"`
3. Processa `Literal(10)` → `inferred_type = "int"`
4. Verifica: `soma` tem `category = Function` ✓
5. `args_enviados = ["int", "int"]`
6. `params_esperados = ["int", "int"]` (extraídos do `parameter_types` do `Symbol`)
7. `args_enviados.len() == params_esperados.len()` ✓ (`2 == 2`)
8. Para cada argumento: `promover_tipo("int", "int") = Some("int")` ✓
9. `node.inferred_type = "int"` (tipo de retorno da função `soma`)

Exemplo de erro — `soma(10)` :

1. `args_enviados = ["int"]`
2. `params_esperados = ["int", "int"]`
3. `args_enviados.len() != params_esperados.len()` (`1 != 2`)
4. `panic!("Funcao 'soma' na linha 10 espera 2 argumentos, mas 1 foram passados")`

Exemplo de erro — `soma(x, "texto")` :

1. `args_enviados = ["int", "string"]`
2. `params_esperados = ["int", "int"]`
3. `args_enviados.len() == params_esperados.len()` ✓
4. Argumento 2: `promover_tipo("string", "int") = None`
5. `panic!("Argumento 2 da funcao 'soma' na linha 10: esperado 'int', obtido 'string'")`

7.20 Return (linhas 682–704)

1. Processar filhos recursivamente
2. Verificar que `current_function_return_type` está definido → `panic!`
3. Se função é `void`:
→ verificar que não há expressão de retorno → `panic!`
4. Se função não é `void`:
→ verificar que há expressão de retorno → `panic!`
→ verificar compatibilidade `tipo_função ↔ tipo_expressão` → `panic!`
5. (não propaga `inferred_type` – `return` não tem tipo)

7.21 Pass-throughs (linhas 676–681)

Nós sem verificação semântica própria: `Inherit`, `AccessSection`, `ParamList`, `Case`, `Default`. Apenas processam filhos recursivamente.

8. Inferência de tipos

8.1 Propagação bottom-up

Cada nó calcula o seu `inferred_type` a partir dos filhos:

```
Identifier  → lookup na tabela de símbolos
Literal     → mapeamento original_token → tipo
BinaryOp    → promover_tipo(esquerda, direita)
UnaryOp     → tipo do operando
Call        → tipo de retorno da função
Assign      → tipo do lado esquerdo
```

8.2 Regras de promoção

```
int + int      → int
int + float    → float
int + double   → double
float + double → double
int + string   → PANIC (incompatível)
bool + int     → PANIC (incompatível)
```

8.3 Operadores relacionais

Todos os operadores relacionais (`==`, `!=`, `<`, `>`, `<=`, `>=`) e lógicos (`&&`, `||`) resultam em `bool`.

9. Gestão de escopos

9.1 O que cria escopos

Apenas `Block` cria escopos. Todos os outros nós usam o padrão `save/restore` no `block_type`.

Isso porque o parser representa escopos como `{ stmts }` $\rightarrow$ `Block`, e qualquer constructo (`if`, `while`, `for`, etc.) tem o seu `body` como filho `Block`.

Exemplo: `if (x > 5) { int temp = x; }`

```

If
├─ BinaryOp (>)
│   └─ Identifier (x)
│       └─ Literal (5)
└─ Block                ← AQUI é que se cria o escopo
    └─ VarDecl
        ├── Type (int)
        ├── Identifier (temp)
        └─ Identifier (x)

```

O `If` não cria escopo — apenas guarda/restaura `block_type`. O `Block` cria o escopo para `temp`.

9.2 Criação de escopo pelo Block

```

AstKind::Block => {
    // 1. Criar novo escopo (filho do actual)
    // 2. Drenar pending_parameters → inserir no novo escopo
    // 3. Processar filhos recursivamente
    // 4. Restaurar escopo anterior
}

```

Exemplo concreto — `int soma(int a, int b) { return a + b; }:`

```

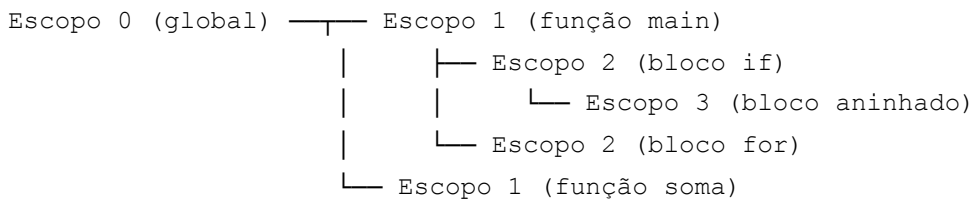
FunctionDecl
├─ Type ("int")
├─ Identifier ("soma")
├─ ParamList
│   ├── Param → Type("int"), Identifier("a")    ← empurra para pending_parameters
│   └─ Param → Type("int"), Identifier("b")    ← empurra para pending_parameters
└─ Block                ← cria escopo, drena pending_parameters
    └─ Return
        └─ BinaryOp ("+")
            ├── Identifier ("a")                ← lookup encontra no escopo do Block
            └─ Identifier ("b")                ← lookup encontra no escopo do Block

```

O que acontece passo a passo:

1. `FunctionDecl` processa os filhos recursivamente
2. `ParamList` → `Param` empurra `Symbol { name: "a", ... }` para `pending_parameters`
3. `ParamList` → `Param` empurra `Symbol { name: "b", ... }` para `pending_parameters`
4. `Block` cria novo escopo (nível + 1)
5. `Block` drena `pending_parameters` → `a` e `b` são inseridos no novo escopo
6. `Return` processa o filho `BinaryOp`
7. `BinaryOp` processa `Identifier("a")` → `lookup` encontra `a` no escopo do `Block`
8. `BinaryOp` processa `Identifier("b")` → `lookup` encontra `b` no escopo do `Block`
9. `Block` restaura `current_scope_idx` para o escopo anterior

9.3 Hierarquia de escopos



Cada escopo mantém `parent_idx` apontando para o escopo pai.

Exemplo em código:

```
int x; // Escopo 0
int main() { // Escopo 1 (filho de 0)
    int a; // Escopo 1
    if (a > 0) { // Escopo 2 (filho de 1)
        int temp; // Escopo 2
        if (temp > 5) { // Escopo 3 (filho de 2)
            int y; // Escopo 3
        }
    }
    for (int i = 0; ...) { // Escopo 2 (filho de 1)
        int val; // Escopo 2
    }
}
```

Vector de escopos resultante:

```
scopes[0] = Scope { level: 0, parent: None, symbols: {x} }
scopes[1] = Scope { level: 1, parent: Some(0), symbols: {main, a} }
scopes[2] = Scope { level: 2, parent: Some(1), symbols: {temp} }
scopes[3] = Scope { level: 3, parent: Some(2), symbols: {y} }
scopes[4] = Scope { level: 2, parent: Some(1), symbols: {i, val} }
```

9.4 Lookup

A busca sobe a cadeia de pais:

Escopo 2 → Escopo 1 → Escopo 0

Se encontrar o nome em qualquer escopo, retorna o símbolo.

9.5 Variable Shadowing

Permitido — uma variável num escopo interno pode ter o mesmo nome que uma num escopo externo. A variável interna "sombra" a externa nesse escopo.

Exemplo:

```

int x = 10;           // Escopo 0: x = Int("int")
int main() {
  int x = 20;         // Escopo 1: x = Int("int") (sombra o x do Escopo 0)
  if (x > 0) {         // usa x do Escopo 1
    int x = 30;       // Escopo 2: x = Int("int") (sombra o x do Escopo 1)
    // aqui, x = 30
  }
  // aqui, x = 20 (o x do Escopo 2 já não existe)
}
// aqui, x = 10 (o x do Escopo 1 já não existe)

```

No relatório:

```

Escopo 0 (nivel 0):  x | tipo: Int("int") | cat: Variable | linha: 1
Escopo 1 (nivel 1):  x | tipo: Int("int") | cat: Variable | linha: 3
Escopo 2 (nivel 2):  x | tipo: Int("int") | cat: Variable | linha: 5

```

Cada `x` existe num escopo diferente — o lookup encontra primeiro o do escopo mais interno.

10. Problemas e soluções

10.1 Parâmetros no escopo correcto

Problema: Os parâmetros são declarados antes do Block do body da função. No parser, a estrutura é:

```

FunctionDecl
  Type("int")
  Identifier("soma")
  ParamList
    Param → Type("int"), Identifier("a")
    Param → Type("int"), Identifier("b")
  Block ← aqui é que os params devem existir
    Return → Identifier("a") + Identifier("b")

```

Solução: `pending_parameters`

1. `Param("a")` empurra `Symbol` para `pending_parameters`
2. `Param("b")` empurra `Symbol` para `pending_parameters`
3. `Block` cria novo escopo e drena `pending_parameters` → `a`, `b` ficam no escopo do `Block`
4. `Identifier("a")` → lookup encontra `"a"` no escopo do `Block` ✓
5. `Identifier("b")` → lookup encontra `"b"` no escopo do `Block` ✓

Sem `pending_parameters`, os parâmetros seriam inseridos no escopo da função (nível 0), onde ficariam visíveis para **todos** os código que não está no body da função — incluindo outras funções no mesmo nível.

10.2 For-loop com escopo isolado

Problema: `for(int i = 0; i < 10; i++)` — `i` só deve ser visível dentro do body do `for`.

Árvore AST do `for`:

```
FOR
├── VAR_DECL                               ← init: int i = 0
│   ├── TYPE (int)
│   ├── IDENTIFIER (i)
│   └── LITERAL (0)
├── BINARY_OP (<)                         ← condition: i < 10
│   ├── IDENTIFIER (i)
│   └── LITERAL (10)
├── UNARY_OP (post++)                     ← increment: i++
│   └── IDENTIFIER (i)
└── BLOCK                                 ← body: { ... }
    └── VarDecl
        ├── Type (int)
        ├── Identifier (temp)
        └── Identifier (i)
```

O que acontece passo a passo:

1. `For` **guarda** `block_type` $\rightarrow$ `BlockType::For`
2. `For` **cria** novo escopo (nível + 1) **antes** de processar o `init`
3. `for_init_mode = true`
4. Processa `init`: `VarDecl` **vê** `for_init_mode == true` $\rightarrow$ **empurra** `Symbol { name: "i" }` para `pending_parameters` em vez de inserir no escopo
5. `for_init_mode = false`
6. `For` **drena** `pending_parameters` $\rightarrow$ `i` é inserido no novo escopo
7. Processa `condition`: `BinaryOp` **processa** `Identifier("i")` $\rightarrow$ `lookup` encontra `i` no escopo do `For`
8. Processa `increment`: `UnaryOp` **processa** `Identifier("i")` $\rightarrow$ `lookup` encontra `i`
9. Processa `body`: `Block` **cria outro** escopo (nível + 2) dentro do escopo do `For`
10. `VarDecl("temp")` **processa** `initializer Identifier("i")` $\rightarrow$ `lookup` encontra `i` no escopo ancestral (`For`)
11. `For` **restaura** `current_scope_idx` e `block_type`

Resultado: `i` existe no escopo do `For` (nível 1), `temp` existe no escopo do `Block` interno (nível 2).

`temp` é visível dentro do `Block`, `i` é visível em todo o `For` (`init`, `condition`, `increment`, `body`).

10.3 For-loop com backtracking (parser)

Problema: `for(int i = 0; ...)` vs `for(i = 0; ...)` — LL(1) não consegue desambiguar porque `int` pode ser início de uma expressão.

Solução (Opção B — backtracking): `st_for_init` guarda a posição do parser, tenta parsear como declaração; se falhar, restaura posição e tenta como expressão.

10.4 `block_type` vs `current_function_return_type`

Problema: `block_type` é usado para `break/continue/constructor`. `current_function_return_type` é usado para `Return`. São independentes porque um `Return` dentro de um `If` dentro de uma função precisa de acessar o tipo da função, não o tipo do `If`.

Solução: Dois campos independentes, ambos com `save/restore` no `FunctionDecl/ConstructorDecl`.

10.5 Literal type disambiguation

Problema: `true` e `1` são ambos representados como `Literal` com token `"true" / "1"`. O parser não tem como saber o tipo semântico.

Solução: `original_token: Option<Token>` preserva a variante original do token para que o analisador semântico distinga `Token::TrueLiteral` de `Token::Inteiro`.

11. Integração com o main

11.1 Processamento (src/main.rs)

```
fn processar_ficheiro(caminho: &Path) -> String {
    let tokens = lexer::tokenizar(&conteudo);
    let ast = analise_sintatica::analisar(tokens);

    // Capturar erros semânticos via panic
    let resultado = panic::catch_unwind(|| {
        analise_semantica::decorar_ast(ast)
    });

    match resultado {
        Ok((ast_decorado, scopes)) => { /* escrever AST + scopes */ }
        Err(_) => { /* escrever erro semântico */ }
    }
}
```

11.2 Captura de erros via panic

O analisador semântico usa `panic!` para todos os erros. O `main.rs` instala um hook temporário que captura a mensagem de erro num `thread_local!`:

```
thread_local! {
    static ULTIMO_ERRO: RefCell<Option<String>> = RefCell::new(None);
}
```

O `panic::catch_unwind` com `AssertUnwindSafe` permite capturar o panic sem abortar o programa.

11.3 Ficheiros processados

Todos os `.cpp` em `src/files/` são processados por ordem alfabética. Cada ficheiro é independente (a AST e scopes são recriados de cada vez).

12. Tabela de símbolos

12.1 Formato no relatório

```
Escopo 0 (nivel 0):
  soma | tipo: Int("int") | cat: Function | linha: 3 | addr: 0 | bytes: 0
    params: ["int", "int"]
  Pessoa | tipo: Int("") | cat: Class | linha: 8 | addr: 0 | bytes: 0
Escopo 1 (nivel 1):
  a | tipo: Int("int") | cat: Parameter | linha: 3 | addr: 0 | bytes: 4
  b | tipo: Int("int") | cat: Parameter | linha: 3 | addr: 4 | bytes: 4
Escopo 2 (nivel 2):
  resultado | tipo: Int("int") | cat: Variable | linha: 5 | addr: 0 | bytes: 4
```

12.2 Endereçamento de memória

Cada escopo mantém `next_memory_offset` que incrementa com o tamanho de cada símbolo:

```
Offset 0: a (int, 4 bytes)
Offset 4: b (int, 4 bytes)
Offset 8: resultado (int, 4 bytes)
```

13. Testes

13.1 Ficheiros de teste (src/files/)

Ficheiro	O que verifica
main.cpp	Programa válido simples
test_var_decl.cpp	Declaração de variáveis
test_undeclared.cpp	Variável não declarada
test_assign.cpp	Atribuição incompatível
test_binary_op.cpp	Operadores binários incompatíveis
test_function.cpp	Declaração e chamada de função
test_array.cpp	Declaração e uso de arrays
test_if.cpp	Condição em if
test_while.cpp	Condição em while

Ficheiro	O que verifica
test_for.cpp	For-loop com declaração e condição
test_for_decl.cpp	For-loop com declaração complexa
test_dowhile.cpp	Condição em do-while
test_break_continue.cpp	Break/Continue em contextos válidos e inválidos
test_class.cpp	Declaração de classes e construtores
test_constructor_error.cpp	Erros de construtor
test_ok.cpp	Programa semanticamente válido completo
test_return_type.cpp	Tipo de retorno incompatível
test_duplicate_var.cpp	Variável declarada duas vezes
test_function_arg_types.cpp	Argumentos incompatíveis em chamada

13.2 Executar testes

```
cargo run
# Verificar relatorio.txt gerado
```

14. Referências

- **Enunciado Fase 3:** `mds/enunciado fase 3.md`
- **Especificação da AST:** `documentos/pp2-analise-sintatica/AST.md`
- **Manual do Utilizador (Parser):** `mds/MU_PARSER.md`
- **Manual do Programador (Parser):** `mds/MP_PARSER.md`